

Assignment Report for Assignment-05

Course and Section	215.01
Assignment Name	Assignment-05
Due Date and Time	08/16/2026 @ 11:59pm
First Name and Last Name	Ahmad Mustafa
SFSU Email Account	amustafa@sfsu.edu
First Name and Last Name of Teammate	[First Name] [Last Name]
SFSU Email Account of Teammate	[SFSU Email Account]

**PART A****Question Description and Analysis:**

Class Design Guidelines

Answer:

In this part, we will discuss a java design guideline called “encapsulation.” Which essentially is the practice of deciding where your data can be accessed.

Understanding the problem:

This question is asking for me to take one of the 8 coding design guidelines from Daniel Liang’s famous java design guidelines and explain it in detail.

The guideline i want to explain is called encapsulation, in which the author suggests that a class should use a private modifier to hide its data from direct access by clients. This makes the class easy to maintain since there would be no interference.

Second bullet point suggests that we only provide a getter method for a variable if you want the data field to be readable and provide a setter method only if you want the data field to be updateable. To my understanding, the author is suggesting to not write these methods unless you are going to use them in your code. Additionally, not adding getter/setter methods ensures no field is accessed/modified unless intended by the programmer.

In order to complete this requirement. I'm planning to create a class named Student. the Student class will contain a student name, GPA, and private address. The address is not supposed to be accessed or modified hence we will not be creating any getter or setter methods for this variable.

There will also be a private class within the Student class to demonstrate the use of private encapsulated classes.

—

Post Programming Reflection:

Let's discuss how I created a new Java class to demonstrate these 2 rules.

I created 3 classes. 1 driver class and 1 Student class and 1 private DanceStudent class nested within the Student class. This nested private class demonstrates the first suggestion to create private classes. I learned in java you can only create private classes if they are nested, you can not begin a class file with a private class.

For the second suggestion, I created 3 variables. 2 of them to be accessed and modified and 1 of them not to be accessed or changed. The first 2 as demonstrated in screenshots are Name and GPA which have getter and setter methods.

The 3rd attribute, address has been made private, unable to be accessed or modified. Since this is a personal detail, it's best left this way.

Screenshots of Outputs and Explanation:

The screenshot shows an IDE with the project 'CSC215Again'. The file 'Java_Design_AhmadMustafa.java' is open, showing a class with a static method 'driver()' and a 'main' method. The 'main' method calls 'driver()', which creates a 'Student' object, sets its name and GPA, and calls the 'dance()' method. The output window shows the execution results.

```

4  *
5  * File: [Java_Design_AhmadMustafa.java]
6  * By: [Ahmad Mustafa]
7  * Date: [08/08/2026]
8  *
9  * Description: The purpose of this code is to demonstrate java class encapsulation.
10 *
11 * *****/
12
13
14 public class Java_Design_AhmadMustafa { & Ahmad Mustafa *
15     private static void driver() { 1 usage & Ahmad Mustafa *
16
17         Student stu1 = new Student(name: "Ahmad Mustafa", GPA: 3.730); // Create object
18         System.out.println(stu1.getName() + " " + stu1.getGPA());
19
20         stu1.setGPA(3.8); // Setter for GPA
21         stu1.setName("Ahmad2 Max"); // Setter for Name
22         System.out.println(stu1.getName() + " " + stu1.getGPA());
23
24         stu1.dance(); // Method from private encapsulated class.
25     }
26
27     public static void main(String[] args) { driver(); }

```

```

/Users/goLa/Library/Java/JavaVirtualMachines/openjdk-24.0.2+12-54/Contents/Home/bin/java -javaagent:/Appl
Ahmad Mustafa 3.73
Ahmad2 Max 3.8
the student dances wohooo
Process finished with exit code 0

```

The screenshot shows the 'Student.java' file in the IDE. It defines a 'Student' class with attributes 'name' and 'GPA', and methods 'getName()', 'setName()', 'getGPA()', 'setGPA()', and 'dance()'. There is also a private static nested class 'DanceStudent' with a static method 'dancer()'.

```

9  public class Student { & Ahmad Mustafa *
10
11     @Contract(pure = true)
12     Student (String name, double GPA) { 1 usage & Ahmad Mustafa
13         this.name = name;
14         this.GPA = GPA;
15     }
16
17     public String getName() { & Ahmad Mustafa
18         return this.name;
19     }
20     public void setName(String name) { & Ahmad Mustafa
21         this.name = name;
22     }
23
24     public double getGPA() { 2 usages & Ahmad Mustafa
25         return this.GPA;
26     }
27     public void setGPA(double GPA) { 1 usage & Ahmad Mustafa
28         this.GPA = GPA;
29     }
30     static void dance() { 1 usage & Ahmad Mustafa
31         DanceStudent.dancer();
32     }
33
34     private class DanceStudent { // Private encapsulated class 1 usage & Ahmad Mustafa *
35         static void dancer() { 1 usage & Ahmad Mustafa
36
37             System.out.println(" the student dances wohooo");
38         }
39     }
40 }
41

```

PART B**Question Description and Analysis:**

Class Relationship, HAS-A, Aggregation and Composition

Answer:

Understanding the Problem:

This problem is asking from us to demonstrate our ability of OOP aggregation and/or composition. As a valid attempt, we must have 5 classes for the program. 1 must be the driver class to run the instance and the other 4 must be related to each other meaning one father class which contains the other 3 subclasses. Then I must also draw a simple diagram of the class relationships (* Attached screenshot below). I'm planning to complete this task by creating 4 classes. 1 Stationary class and then as subclasses I want a pencil, eraser and sharpener class.

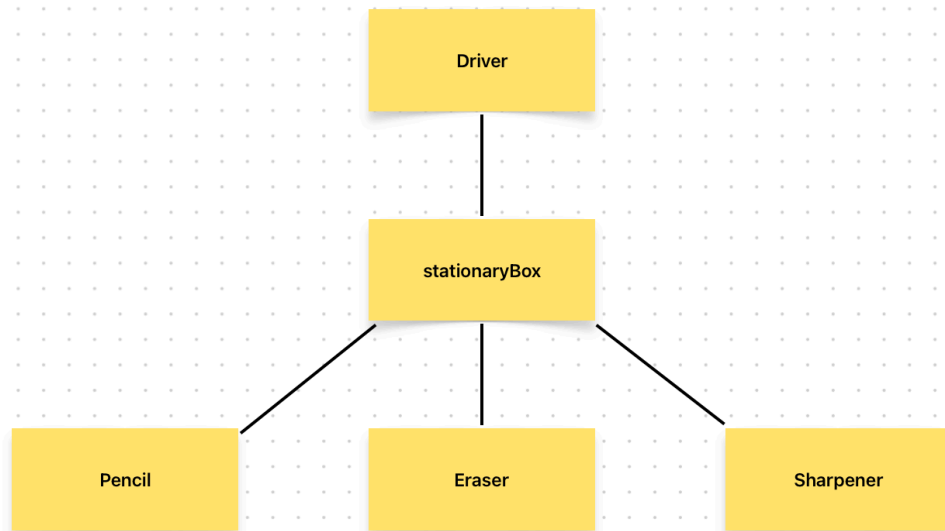
—

Post Programming Reflection:

The program compiles successfully and it was executed as I had imagined. I was able to meet all client requirements and I did that by creating 5 classes. 1 Class on the top of the flowchart is the driver class. Then one level below that is the stationaryBox class. Then one level below that is 3 subclasses; pencil, eraser and sharpener. This concludes all requirements from the client into a successful execution of Has-A relationships.

****Code Attached in the assignment file.**

Screenshots of Outputs and Explanation:



```

9      description: The purpose of this code is to demonstrate OOP Aggregation.
10
11      *****/
12
13
14  public class OOP_ClassRelationship_HAS_A_AhmadMustafa {
15      // Ahmad Mustafa *
16
17      public static void driver() {
18          // Ahmad Mustafa *
19
20          pencil pen1 = new pencil(); // pencil object
21          eraser era1 = new eraser(); // eraser object
22          sharpener sha1 = new sharpener(); // sharpener object
23
24          Object[] obj = {pen1, era1, sha1}; //Object array for aggregation.
25
26          stationaryBox sta1 = new stationaryBox( color: "Black", obj); //Stationary object with aggregated object array.
27          sta1.displayInfo();
28      }
29
30      public static void main(String[] args) { driver(); }
31  }

```

Run OOP_ClassRelationship_HAS_A_AhmadMustafa

```

This Black stationary box contains:
A RAW pencil
A White eraser
A RAW sharpener

Process finished with exit code 0

```

PART C

Question Description and Analysis:

Class Relationship, Inheritance IS-A

Answer:

The client is requesting for our team to demonstrate our understanding of Is-A relationships.

They are asking for us to define 4 classes in total. 1 driver class and a 3 layered class structure

(I.E Class→Subclass→Subclass). Among the requirements is to build a flowchart diagram and

share it with the client (*Screenshot attached below). I am planning to complete this assignment by creating a Person class, who is also a Student and who is also a Computer Science Student. This would entail a 3 layer class is-a relationship inheritance.

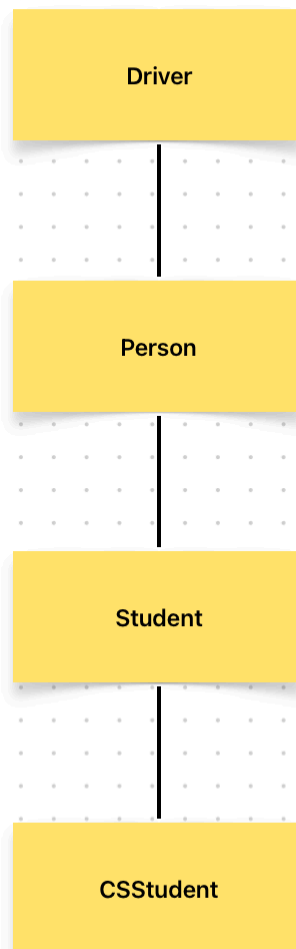
Post Programming Reflection:

All client requirements have been met and a demonstration has been coded as well. Also a flowchart has been attached in the screenshots below. To explain further, the granchild class is the CSSStudent who is-A Student who is-A Person. This explains the 3 layers of a person's identity, hence separate classes. Then also a driver method on top of this structure to run the program. This in other words is also commonly known as inheritance in Java OOP. To further clarify, the CSSStudent would inherit attributes and methods from the Student class and the person class since a cs student is also naturally a student and a person. My program demonstrates this relationship and further supports this explanation.

**** Code Attached in the file**

--

Screenshots of Outputs and Explanation:



The screenshot shows an IDE window for a project named 'CSC215Again'. The project structure on the left includes a 'src' directory with subdirectories 'ASMT01' through 'ASMT06', and 'partA', 'partB', and 'partC'. The 'ASMT05' directory contains the files 'CSStudent', 'OOP_ClassRelationship_Inheritance_IS_A_AhmadMustafa', 'Person', and 'Student'. The 'ASMT05.md' file is open in the editor, showing the following Java code:

```
7  * Date: [08/08/2026]
8  *
9  * Description: The purpose of this code is to demonstrate OOP Inheritance.
10 *
11 * *****/
12
13
14 public class OOP_ClassRelationship_Inheritance_IS_A_AhmadMustafa { & Ahmad Mustafa *
15
16     public static void driver() { & Ahmad Mustafa
17
18         CSStudent css1 = new CSStudent(); //Grandchild class
19         css1.hello();
20
21         Student stu1 = new Student(); //Child class
22         stu1.hello();
23
24         Person per1 = new Person(); //Father class
25         per1.hello();
26
27         Person per2; // Object declared as Person
28         per2 = new Student(); // Object initialised as a Student
29         per2.hello(); //Object defined as Student
30
31     }
32
33     public static void main(String[] args) { driver(); }
34 }
35
```

The 'Run' window at the bottom shows the execution output:

```
/Users/go1a/Library/Java/JavaVirtualMachines/openjdk-24.0.2+12-54/Contents/Home/bin/java -javaagent:/Appl
Hello I am a CS Student
Hello I am a Student
Hello I am a Person
Hello I am a Student
Process finished with exit code 0
```

The status bar at the bottom indicates the current file is 'OOP_ClassRelationship_Inheritance_IS_A_AhmadMustafa' and the cursor is at line 7, column 16, in the 'main' method.